A Source-Agnostic AI-Powered Smart City Observatory: Self-Generating, Self-Healing, and Self-Deploying Data Wrappers in Production

Anonymous Author(s)

***Abstract*—A smart city observatory is only as useful as the breadth of sources it can ingest, yet most indicator platforms hard-wire a handful of feeds and pay an engineer to maintain a wrapper for each. We present the Observatory, a production smart city observatory whose ingestion layer is source-agnostic: any REST API, spreadsheet, or CSV export becomes a normalized indicator time series with no source-specific code written by a human. The platform treats each source's data wrapper as the managed element of an autonomic manager and runs three control loops over it. Self-generation plans a new artifact: a large language model fills a fixed extraction template from source metadata and a data sample, probing live endpoints before it commits to extraction logic. Self-deployment executes it: wrappers run as isolated, supervised, checkpointed processes that the platform launches and relaunches on its own. Self-healing closes the loop: a monitor re-invokes the model with the production crash trace and redeploys a repaired wrapper. We cast the three loops in MAPE-K terms over one shared knowledge base and report a 149-day deployment: 396 wrappers across API, XLSX, and CSV sources ingested 164,322 data points; 43 (10.9 %) crashed and were repaired automatically, with the crash recurring for only one of them thereafter, at a median of 33.4 s per repair; and the platform runs in about 1.5 GiB of memory. We read the result as a scope of control: source heterogeneity and code defects fall inside the manager's authority, while quota, credential, and misconfiguration faults are environmental and fall outside it.**

***Index Terms*—smart city observatory, source-agnostic data ingestion, autonomic computing, self-healing systems, LLM code generation, ETL automation, smart tourism**

I. Introduction

Smart city and destination observatories promise a single pane of glass over heterogeneous urban indicators: air and water quality, mobility, energy, overnight stays, cultural participation. The promise is bounded in practice by the ingestion layer. Each indicator is published by a different statistical office, municipality, or sector association, as a REST API, an Excel workbook, or an ad hoc CSV export, with its own schema, date format, language, and failure modes. The classical answer is one extraction wrapper per source, but writing and maintaining those wrappers is the cost that has limited mediator-based data integration since the 1990s [1]–[3]. A platform that can only afford a fixed set of wrappers is, in effect, not an observatory of its city but of the few sources someone had time to integrate.

We call an ingestion layer *source-agnostic* when adding a new source costs no source-specific engineering: a domain user registers the source through a web form, and the platform produces, runs, and maintains the wrapper on its own. This paper describes the Observatory (anonymized for review), a production smart city observatory for a European region in which source-agnostic ingestion is realized by treating each wrapper as an *autonomic* element in the sense of Kephart and Chess [4]: a self-managing component with a closed control loop over its own lifecycle. The actuator that closes the loop is unusual. Rather than tuning the parameters of a fixed component, a large language model (LLM) writes the component's code, and rewrites it when it breaks.

Every wrapper has three self-* properties, which we cast as the monitor–analyze–plan–execute loop over shared knowledge (MAPE-K) of the autonomic-computing blueprint [5], [6]. *Self-generation* (Section IV-B) plans a new artifact: an LLM fills a fixed template from indicator metadata and a data sample, probing live APIs through a controlled fetch tool before writing code. *Self-deployment* (Section IV-C) is the execute step that needs no operator: wrappers run as isolated processes the platform spawns, supervises, checkpoints, and resumes from per-wrapper watermarks after restarts. *Self-healing* (Section IV-D) is the full loop on a running artifact: a monitor detects a crash, the failure is classified, the model re-plans from the captured trace, and the repaired wrapper is redeployed.

This paper makes three contributions. First, we give a source-agnostic observatory architecture and the contract that makes it work: any source is reduced to a stream of normalized $(\text{timestamp}, \text{value})$ points by a generated wrapper that the platform owns end to end (Section III). Second, we unify the three self-* properties as one autonomic manager over a shared knowledge base, separating their MAPE-K roles and relating each to prior work in autonomic computing, LLM code generation and repair, and wrapper maintenance (Section IV). Third, we report measurements from a real 149-day deployment rather than a benchmark, including a pre-repair baseline (Section V-B) and a fault analysis that draws the manager's scope of control precisely (Section V-D). Anonymized source mirrors will be linked here for review.

II. Background and Related Work

A. Smart city and tourism observatories

Smart tourism research calls for destination-level data infrastructures [7], [8], indicator schemes such as ETIS define what such platforms should measure [9], and implementation studies report uneven adoption [10] and warehouse- or survey-based dashboards [11]. Studies of smart destination programs repeatedly identify data availability for sustained indicator monitoring as the bottleneck [12], [13]. Reported platforms rarely document the ingestion layer; we argue that source-agnostic ingestion is what decides how many indicators an observatory can sustain, and we document one in production. Although our deployment is a tourism observatory, the ingestion pattern is domain-independent: it assumes only heterogeneous sources and a common time-series target.

B. Autonomic computing and self-* systems

The autonomic computing vision frames self-managing components with closed control loops as the answer to operational complexity [4], organized as a monitor–analyze–plan–execute loop over a shared knowledge base (MAPE-K) [5], with self-configuration, self-healing, and self-optimization as canonical properties surveyed in [6]. the Observatory instantiates the blueprint for data wrappers and contributes a different actuator: where classical autonomic managers plan over a fixed component's parameters, ours plans the component's source code through an LLM, so self-configuration becomes code synthesis and self-healing becomes code repair.

C. LLMs for code generation, repair, and extraction

LLMs generate working programs from natural language [14], and program repair is increasingly LLM-based [15]. Feedback loops improve generated code: Self-Refine [16], Reflexion [17], and Self-Debugging [18], though self-repair gains are not uniform [19]. On the data side, Evaporate showed that generated extraction code amortizes inference cost across documents [20], AutoScraper generates reusable scrapers [21], and prompted [22] and instruction-tuned [23] models perform classical data preprocessing. In MAPE-K terms our planner reuses these techniques, but its sensor is a production exit code rather than a unit test, and its plan rewrites the whole managed artifact rather than a single faulting line.

D. Wrapper induction, maintenance, and ETL automation

Wrapper induction learned extraction rules from labeled examples [2]; unsupervised systems inferred them from page regularities [24]; surveys review the field [25], [26]. The maintenance problem, detecting breakage and re-inducing the wrapper, was recognized early [3], [27]. ETL has been modeled rather than synthesized [28], [29], with automation of schema matching [30], curation [31], and wrangling [32]. the Observatory replaces both induction and re-induction with LLM code generation, extending coverage beyond template pages to arbitrary APIs and spreadsheets, and automates the narrow but recurrent ETL slice indicator platforms need: single-series extraction into one normalized time-series schema.

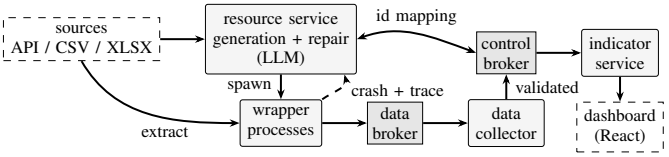


Fig. 1. The Observatory ingestion pipeline. The resource service generates, deploys, and repairs wrappers; extracted points flow through two brokers to the indicator store and the dashboard.

III. A Source-Agnostic Observatory

The Observatory is a microservice platform: containerized user, knowledge-base, data-collector, resource, and indicator services (Python, FastAPI) behind an nginx gateway, plus a React dashboard. Two RabbitMQ brokers separate the control plane from the data plane; the resource and indicator services each own a MongoDB store. Figure 1 shows the pipeline.

The property that makes ingestion source-agnostic is a narrow contract. A wrapper, whatever its source, publishes batches of points $\{(x, y)\}$ with x an ISO 8601 timestamp and y a float; everything upstream of that contract is generated per source, and everything downstream is fixed, shared infrastructure. A domain user registers a source in a web wizard, either uploading a CSV/XLSX file or configuring an API endpoint with its authentication, and supplies indicator metadata (name, domain, unit, periodicity, description). The resource service then owns the wrapper for its whole lifecycle. The data collector validates each message against the contract, caches the latest per wrapper, and forwards valid points to the control broker, where the resource service maps wrapper identifiers to persistent resources and the indicator service stores, merges, and serves the series. Source type enters only as a parameter to one generation procedure, not as a separate codebase per format, which is what lets the same machinery absorb a new kind of source.

IV. Self-* Wrappers as an Autonomic Manager

We model each wrapper as the managed element of one autonomic manager. The element is the generated artifact together with its runtime state: the wrapper's source code, its execution watermarks, its retry counter, and its status. Over this element the platform runs the MAPE-K loop [5]: it *monitors* process state, *analyzes* terminations, *plans* a new or corrected artifact, and *executes* the plan by deploying it, all against a shared *knowledge* base (Table I). Figure 2 shows the three self-* loops this decomposition produces. The actuator across all of them is an LLM that writes and rewrites code, paired with a process supervisor that deploys it.

A. Shared knowledge

The knowledge the loops read and write is what makes them one manager rather than three mechanisms. It has a static part and a runtime part. The static part is the fixed wrapper template, which is the contract of Section III expressed as

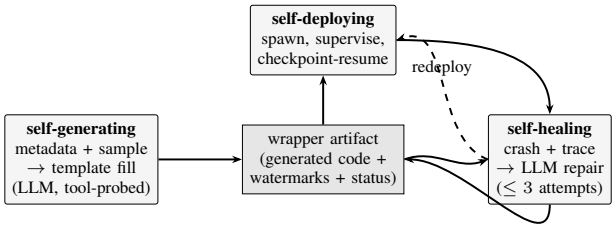


Fig. 2. The autonomic wrapper manager. Three self-* loops act on one shared artifact; the actuator is an LLM that writes and rewrites code, plus a process supervisor that deploys it.

TABLE I
THE AUTONOMIC LOOP MAPPED ONTO THE PLATFORM. ONE MANAGER REALIZES ALL THREE SELF-* PROPERTIES; THE ACTUATOR IN THE *plan* STEP IS AN LLM.

MAPE-K stage	Realization in the Observatory
Monitor	30 s poll of each wrapper subprocess; exit codes, watchdog terminations, last-data timestamps
Analyze	classify a termination: code crash (exit 1) vs. idle watchdog timeout vs. provider/quota error; only crashes route to repair
Plan	LLM generates or repairs the whole wrapper file, tool-probing live APIs, under the fixed template and hard rules
Execute	static validation, then spawn or relaunch the isolated process, resuming from watermarks; semaphore-bounded
Knowledge	fixed template and the rule list distilled from past failures; per-wrapper watermarks, retry counter, execution log, status

code, plus a list of fourteen hard rules distilled from failures seen during development: never modify the historical-backfill driver, emit no synthetic data, filter date ranges inclusively, keep timezones consistent, and parse non-English dates such as localized month names. These bound what the planner may produce. The runtime part is per-wrapper: two watermarks (the oldest and newest timestamps ever sent) and a phase, a retry counter in $[0, 3]$, a timestamped execution log, and the persisted wrapper document holding metadata, source configuration, current code, and status. Self-generation writes the initial code, self-deployment reads and updates the watermarks, and self-healing reads the code and the log and writes a new revision; the rule list, in particular, accumulates the lessons of past failures and constrains every future plan.

B. Self-generation: planning a new artifact

Generation is constrained, not free-form. The model customizes a fixed wrapper class whose variable methods are extraction (`fetch_external_data`) and, for API sources, a windowed fetch and a polling interval; the backfill driver, transport, batching, and run loop are fixed library code the prompt forbids it to touch (Figure 4). Jinja2 assembles the prompt from the indicator metadata, the source configuration, a short data sample (20 CSV lines, 15 rows per workbook sheet with the sheet names, or one captured API response), the target `{x,y}` schema, the template, per-source-type guidance, and

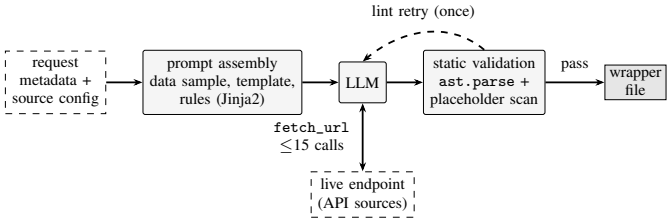


Fig. 3. Self-generation. The prompt combines metadata, a data sample, the fixed template, and hard rules; for API sources the model probes the live endpoint before writing code; the output must clear a static gate, with one retry, before launch.

```
class Wrapper:
    # fixed: transport, backfill, run loop (do not edit)
    async def fetch_historical_data(self):
        return await self.backfill(
            self.fetch_date_range, self.send_to_queue)

    # variable: the model fills only these
    async def fetch_external_data(self):
        """return [{'x': <ISO-8601>, 'y': <float>}, ...]"""
        ... # source-specific extraction
    async def fetch_date_range(self, start, end):
        ... # API only: windowed fetch
    def get_interval_seconds(self):
        ... # API only: poll period
```

Fig. 4. The wrapper contract (abridged). The model customizes only the variable methods; backfill, transport, and scheduling are fixed. Confining generation to extraction logic is what makes source type a parameter rather than a codebase.

the rule list. For API sources the call also offers a sandboxed `fetch_url` tool that returns up to 2500 characters of a live response (Figure 3). The model may invoke it up to fifteen times, reading real response nesting, field names, and honored date filters instead of guessing from the URL; authentication is folded into request headers by the service from the declared scheme, not handed to the model. A static gate runs before launch: the file is parsed and scanned for unfilled placeholders, one failure triggers a single lint-retry prompt, and a second marks the wrapper `error`. Nothing is test-run before launch; runtime faults are left to self-healing.

C. Self-deployment: execute without an operator

A generated wrapper reaches production with no human action. The resource service launches each wrapper as its own operating-system process, handing it the wrapper identifier and source configuration as arguments and the broker URL through the environment. A semaphore caps how many run at once (100 by default), and a per-process watchdog (3600 s by default) bounds any single run. File wrappers run once and exit; API wrappers backfill history in adaptive batches and then poll at the cadence implied by the indicator periodicity (hourly becomes 3600 s, daily 86 400 s), looping inside the process with no external scheduler. Points leave in messages of at most 1000. A monitor samples each process's state every 30 s. Each wrapper also carries a low and a high watermark, the oldest and newest timestamps it has sent; on relaunch it continues from the high mark instead of re-pulling, so a restart neither duplicates nor drops points, with the resume position passed

in through the environment. One level up, a container auto-updater rolls new service images into production without a manual deploy. Self-deployment is the execute step that keeps the other two loops unattended: generation hands the manager a process to supervise, and healing hands it a replacement to relaunch and resume.

D. Self-healing: the full loop on a live artifact

Self-healing exercises every MAPE-K stage. The 30 s poll reads each process's exit code (*monitor*). A nonzero exit, always code 1 in practice, is told apart from an idle watchdog termination and from a generation-time provider error, and only the first routes to repair (*analyze*). The planner is an LLM call given the whole crashed file, the captured stderr trace, and the last twenty stdout lines, and asked to return a complete corrected file that keeps the class structure and fixes that traceback (*plan*); for API wrappers it may re-probe the endpoint through the same `fetch_url` tool, up to ten calls. The reply clears the same static gate as initial generation, overwrites the file, increments the retry counter, and the process is restarted and resumed from its watermarks (*execute*). At most three repairs are allowed per wrapper, after which it is left `error` for an operator. Repairing a managed element rather than patching a diff gives the loop two features: its sensor is a real production exit code, possibly weeks after generation, so feedback reflects how the source actually behaves; and because the planner re-emits the entire wrapper, a fix can reshape the whole extraction path rather than edit the single offending statement [18], [19].

E. Scope of control

Source-agnosticism is then an emergent property of the manager rather than a separate mechanism. The same knowledge and the same loops handle API, XLSX, and CSV sources because source type is a parameter to one template, so a new kind of source is absorbed by generating, deploying, and repairing its wrapper without a human writing code. This also draws the manager's authority. Faults that lie in the managed element, malformed extraction logic and source heterogeneity, are inside its control and are removed by generation and repair; faults in the environment around the element, exhausted model quota, wrong credentials, and limits misconfigured for a source's cadence, are outside it. Section V quantifies both sides of that line.

V. Production Deployment and Metrics

We analyze a 149-day deployment (January 13, 2026 to June 11, 2026) ending in a full snapshot of platform state: the wrapper collection (396 documents with generated code and execution logs), per-wrapper logs, the resource and indicator stores, service logs, and container statistics. All figures below are computed by a script that accompanies the submission. At the snapshot the platform occupied 17 containers using 2,082 MiB of memory at idle, about 1.5 GiB once the optional single-sign-on container is excluded; the resource service, which runs every wrapper process, was the largest single container at 682 MiB.

TABLE II
Wrapper outcomes by source type after 149 days. *Recovered* = entered the repair loop and ended in a non-error state. Heterogeneity is absorbed by the same three loops regardless of source type.

Type	n	Completed	Crashed	Recovered	Points
XLSX	188	104 (55.3%)	26	26	53,530
API	184	52 (28.3%)[a]	14	10	79,411
CSV	24	14 (58.3%)	3	3	31,381
All	396	170 (42.9%)	43	39	164,322

[a]API wrappers poll continuously and rarely complete on their own; 54 ended in `error` after execution-watchdog termination while idle (Section V-D), not from a code defect.

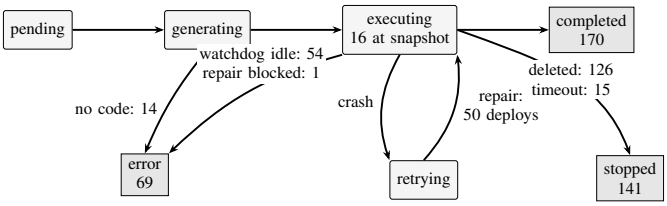


Fig. 5. Wrapper status lifecycle with production counts ($n = 396$). The crash–repair cycle covered 43 wrappers and 50 repairs; most `error` states are watchdog terminations of idle API pollers, not code defects (Section V-D).

A. Generation and coverage

The manager generated 396 wrappers, 188 from XLSX, 184 from API, and 24 from CSV sources, spread across the three domains (environment 203, economy 139, sociocultural 54), and they ingested 164,322 data points (XLSX 53,530, API 79,411, CSV 31,381). The distribution is skewed: the median wrapper sent 27.5 points and the 90th percentile 967, while 284 of 396 delivered at least one point. Median latency from a generation request to the first process start was 26.8 s (90th percentile 109.8 s over 331 wrappers), covering queueing, data sampling, the model call, and validation. Downstream, of 179 active indicators 118 have a linked source and 111 are populated end to end; the largest active series is an hourly air-quality feed of 20,069 points, and historical backfill carries some series back to the year 2000. Table II breaks outcomes down by source type, and Figure 5 shows the status lifecycle with production counts.

B. Self-healing effectiveness

Of the generated wrappers, 43 (10.9%) raised an uncaught exception at runtime and were handed to the repair loop, which issued 50 redeployments; per-wrapper repair depth was 37 at one attempt, 5 at two, and 1 at three (by source type, 26 were XLSX, 14 API, and 3 CSV). What counts as recovery differs by yardstick, and the yardsticks disagree. The crash recurred after the final repair for just one of the 43, leaving 42 crash-free at the wrapper level. By final status, 39 of 43 (90.7%) ended outside an error state. By output, 32 went on to emit data after their last repair, 20,351 points, an eighth of the corpus. The lone wrapper that used all three repairs never stabilized; it carries a non-error final status only because its resource

was later removed, so we do not read the deepest-retry case as a win. Repairs were quick and hands-off: a median 33.4 s from crash to redeploy (interquartile range 24.2–50.8 s, 90th percentile 60.5 s, maximum 80.4 s across the 50 events), of which up to 30 s is detection lag. The median failure landed 16.8 s into a run, inside the fetch-and-parse phase, and none struck while a poller sat idle.

The window also supplies an unplanned control. No repair loop ran for the first 65 days (it enters the logs on 2026-03-19), and 36 crashes of this exit class over that span were left to die, which we treat as the pre-automation baseline. Two timeline effects temper the healing counts. Thirty-one of the 50 repair events fall on one day, when a credential fix triggered a redeploy that relaunched every wrapper at once, so the events are not independent. And because each relaunch overwrites the stderr file, a triggering traceback survives only when the final run also crashed; for wrappers that recovered we read the cause from the fixes left in their code instead. Those fixes are diagnostic: a comment or guard naming a repaired defect appears in 7 of the 43 crashed wrappers and in none of the 353 that never crashed, and timezone fixes in 4 of 43 against 1 of 353. Table III adds the repair-depth breakdown to the fault analysis.

TABLE III
ROOT CAUSES OF THE 69 WRAPPERS WITH FINAL STATUS `error`, AND OUTCOMES OF THE SELF-HEALING LOOP (43 CRASHED WRAPPERS, 50 AUTOMATIC REPAIRS).

(a) Root causes of `error` status ($n = 69$)

Stage	Root cause	n
Generation	Model daily free-tier quota (HTTP 429)	9
Generation	Model per-request rate limit (HTTP 429)	1
Generation	Model provider overloaded (HTTP 503)	1
Generation	Service restart during generation	3
Runtime	Execution watchdog (3600 s), API pollers[a]	54
Runtime	Crash; repair blocked by LLM quota (429)	1
	Total	69

(b) Self-healing outcomes (43 wrappers, all crashes exit 1)

Healed after 1 repair (crash never recurred)	37
Healed after 2 repairs	5
Still crashing after 3 repairs (retries exhausted)	1
Final status: completed / stopped / executing	28 / 8 / 3
Final status: error (later watchdog timeout)[b]	4
Wrapper-level healing success	42/43
Median crash-to-redeploy repair latency	33.4 s

[a]No traceback in stderr (53/54 empty, 1 non-fatal warning); processes were killed while idle-waiting between polling cycles because the 1 h watchdog is shorter than daily/monthly polling intervals. 48 of the 54 had already delivered data (52,372 points). [b]All four were repaired successfully and crashed no further; the subsequent `error` status stems from the watchdog limit, not from the repaired code.

C. Repair case studies

Four wrappers illustrate the range. The first two are inferred from the repaired code, the third quotes its original exception verbatim, and the fourth has a preserved trace because it never healed.

1) Naive-versus-aware datetime comparison (API, healed): An annual urban-waste API wrapper crashed 82 s into a backfill when it compared timezone-aware source dates against naive window bounds. The repair normalizes both bounds to naive datetimes before comparing and logs instead of raising on an empty fetch. Redeployed in 34.7 s, the wrapper completed with 596 points.

2) Duplicated date parameters (API, healed): An air-quality wrapper's configured endpoint URL already carried a date window, and the generated code tacked on a second, so the server answered HTTP 400. The repair strips every spelling of the start/end parameters from the configured URL before applying its own window and absorbs residual 400s instead of crashing. Redeployed in 51.3 s, it was still streaming at the snapshot (395 points).

3) Header token inside data rows (XLSX, healed): A cultural-heritage spreadsheet repeated its Portuguese year header (`"Ano"`) inside the data, and the wrapper crashed on `int("Ano")`; the repaired file quotes the exception verbatim, so the cause is certain. The repair replaces raw integer casts with coercing conversion and drops non-numeric rows, and three neighbouring wrappers picked up the identical fix the same night. Redeployed in 61.6 s, it completed.

4) Absent second column (CSV, not healed): A search-interest export parsed into a single column, so no (time, value) pair existed. Three repairs (27.0, 33.0, and 51.8 s) added delimiter sniffing and bad-line skipping but could not invent a column, and the loop gave up. A sibling wrapper on a two-column variant of the same export healed on its first attempt. The manager repairs code; it cannot manufacture data that the source does not contain.

D. Faults inside and outside the manager

Table III breaks the 69 error-status wrappers down by root cause, and almost none point to faulty generated code. A first group of 14 never produced code at all: 9 hit the model's daily free-tier quota, one a per-request rate limit, one a provider overload, and three were cut off by a service restart. A second and larger group, 54 wrappers, are API pollers the 3600 s watchdog killed between polling cycles, 53 of them without leaving a traceback; the watchdog is simply tighter than these sources' daily or monthly cadence, a configuration mismatch rather than a defect, and 48 had already shipped 52,372 points before the cut-off. One wrapper crashed and then could not be repaired because the repair call itself hit the quota wall; this is the single error-status wrapper that reflects generated code whose repair never deployed. The division is the scope of control of Section IV-E in numbers: malformed extraction code and format quirks leave the operator's queue, while exhausted quota, wrong credentials, and limits set for the wrong cadence stay outside the managed element and stay manual.

E. Pipeline integrity and throughput

Points attenuate predictably along the pipeline: 164,322 sent by wrappers become 148,131 raw and 106,442 consolidated points in the resource store, and 100,550 points in the indicator

store that merge into series totaling 84,431 points. The attrition reflects deleted resources (198 of 396), regeneration that clears prior data, points arriving for resources with no linked indicator, and timestamp-level deduplication at merge time. Integrity held where it was checked: the data collector logged no schema rejections over 30 days and only 18 duplicate-timestamp warnings, which it resolved by averaging; in an 8.3-hour control-plane window the resource service published 153 messages and the indicator service consumed exactly 153, with no loss across the broker; and the front-door gateway served 55,086 requests over 20 days (about 2,747 per day, 80.3% returning HTTP 200). Resumption worked in production: a container recreation a few hours before the snapshot forced all 16 then-running wrappers to restart, 15 resuming from their stored watermarks while the one without a persisted lower mark replayed its history in full.

VI. Discussion

Reading the deployment through the autonomic manager explains both what worked and where it stops. The manager keeps source-specific code out of human hands by bounding generation with a fixed contract, grounding the plan in observed rather than assumed source behavior, and triggering repair on a real production trace. Its limits are exactly the faults outside the managed element: the only error that reflects undeployed code repair sat behind a quota wall, and the only crash the loop could not fix involved data that did not exist.

Several limitations qualify the measurements. They are observational, from one deployment, one model family, and one region, with no contemporaneous control arm; the pre-loop period differs in source mix and load, and the crash events are clustered rather than independent. We cannot compare against hand-written wrappers on the same sources, which were never built. Recovery counts wrappers restored to a working state, not the semantic correctness of every extracted value, since validation covers schema rather than ground truth, and for healed wrappers the trigger is inferred from repair artifacts because the relaunch overwrites the trace. The platform pins the model through a floating version alias, so the underlying model may have changed during the window; generation latency includes queueing and is an upper bound on model time; the memory footprint is an idle measurement; and watermark-based coverage spans are computable for only the 11 wrappers whose lower mark was persisted.

VII. Conclusion

A smart city observatory becomes source-agnostic when its wrappers manage themselves. Casting each wrapper as the managed element of one MAPE-K manager unifies three properties that are usually treated separately: the manager plans a wrapper from metadata and a sample, executes it as a supervised process that resumes from checkpoints, and re-plans it from production traces when it breaks, all over a shared knowledge base of a fixed contract and accumulated repair rules. In 149 days the manager sustained 396 wrappers and 164,322 points; of 43 that crashed, all but one were left crash-free by at most three repairs, at a median of 33.4 s each and with no operator involvement. The faults it could not absorb were environmental, which points the way forward: semantic validation of extracted values, and an autonomic treatment of quota and credential faults as managed state in their own right rather than as work left on the operator's desk.

References

[1] G. Wiederhold, "Mediators in the architecture of future information systems," *Computer*, vol. 25, no. 3, pp. 38–49, 1992.

[2] N. Kushmerick, "Wrapper induction: Efficiency and expressiveness," *Artificial Intelligence*, vol. 118, no. 1–2, pp. 15–68, 2000.

[3] K. Lerman, S. N. Minton, and C. A. Knoblock, "Wrapper maintenance: A machine learning approach," *Journal of Artificial Intelligence Research*, vol. 18, pp. 149–181, 2003.

[4] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[5] IBM Corporation, "An architectural blueprint for autonomic computing," IBM, White Paper, 2005.

[6] M. C. Huebscher and J. A. McCann, "A survey of autonomic computing—degrees, models, and applications," *ACM Computing Surveys*, vol. 40, no. 3, pp. 1–28, 2008.

[7] U. Gretzel, M. Sigala, Z. Xiang, and C. Koo, "Smart tourism: foundations and developments," *Electronic Markets*, vol. 25, no. 3, pp. 179–188, 2015.

[8] D. Buhalis and A. Amaranggana, "Smart tourism destinations," in *Information and Communication Technologies in Tourism 2014*, Z. Xiang and I. Tussyadiah, Eds. Cham: Springer International Publishing, 2013, pp. 553–564.

[9] European Commission, Directorate-General for Internal Market, Industry, Entrepreneurship and SMEs, "The European Tourism Indicator System: ETIS toolkit for sustainable destination management," Publications Office of the European Union, Luxembourg, Tech. Rep., 2016.

[10] P. Modica, A. Capocchi, I. Foroni, and M. Zenga, "An assessment of the implementation of the European Tourism Indicator System for sustainable destinations in Italy," *Sustainability*, vol. 10, no. 9, p. 3160, 2018.

[11] M. Fuchs, W. Höpken, and M. Lexhagen, "Big data analytics for knowledge generation in tourism destinations – A case from Sweden," *Journal of Destination Marketing & Management*, vol. 3, no. 4, pp. 198–209, 2014.

[12] J. A. Ivars-Baidal, M. A. Celdrán-Bernabeu, F. Femenia-Serra, J. F. Perles-Ribes, and D. Giner-Sánchez, "Measuring the progress of smart destinations: The use of indicators as a management tool," *Journal of Destination Marketing & Management*, vol. 19, p. 100531, 2021.

[13] J. A. Ivars-Baidal, J. F. Vera-Rebollo, J. Perles-Ribes, F. Femenia-Serra, and M. A. Celdrán-Bernabeu, "Sustainable tourism indicators: what's new within the smart city/destination approach?" *Journal of Sustainable Tourism*, vol. 31, no. 7, pp. 1556–1582, 2023.

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[15] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1482–1494. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00129

[16] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[17] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[18] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=KuPixIqPiq

[19] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=y0GJXRungR

[20] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023.

[21] W. Huang, Z. Gu, C. Peng, J. Liang, Z. Li, Y. Xiao, L. Wen, and Z. Chen, "AutoScraper: A progressive understanding web agent for web scraper generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 2371–2389. [Online]. Available: https://aclanthology.org/2024.emnlp-main.141/

[22] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[23] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8754–8782.

[24] V. Crescenzi, G. Mecca, and P. Merialdo, "RoadRunner: Towards automatic data extraction from large web sites," in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2001, pp. 109–118.

[25] A. H. F. Laender, B. A. Ribeiro-Neto, A. S. da Silva, and J. S. Teixeira, "A brief survey of web data extraction tools," *SIGMOD Record*, vol. 31, no. 2, pp. 84–93, 2002.

[26] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, "Web data extraction, applications and techniques: A survey," *Knowledge-Based Systems*, vol. 70, pp. 301–323, 2014.

[27] N. Kushmerick, "Wrapper verification," *World Wide Web*, vol. 3, no. 2, pp. 79–94, 2000.

[28] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proceedings of the 5th ACM International Workshop on Data Warehousing and OLAP (DOLAP '02)*. McLean, VA, USA: ACM, 2002, pp. 14–21.

[29] P. Vassiliadis, "A survey of Extract–Transform–Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[30] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[31] M. Stonebraker, D. Bruckner, I. F. Ilyas, G. Beskales, M. Cherniack, S. B. Zdonik, A. Pagan, and S. Xu, "Data curation at scale: The Data Tamer system," in *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org, 2013. [Online]. Available: http://cidrdb.org/cidr2013/Papers/CIDR13_Paper28.pdf

[32] T. Furche, G. Gottlob, L. Libkin, G. Orsi, and N. W. Paton, "Data wrangling for big data: Challenges and opportunities," in *Proceedings of the 19th International Conference on Extending Database Technology (EDBT 2016)*. Bordeaux, France: OpenProceedings.org, 2016, pp. 473–478.